

第十七章 软件架构的腐蚀和对策

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第17章 软件架构的腐蚀和对策

- 17.1 引言
- 17.2 软件架构腐蚀的含义
- 17.3 软件架构腐蚀的预防控制策略
- 17.4 软件架构实践中面临的主要威胁及其对策
- 17.5 小结

17.1 引言 (I)

- 软件架构是软件系统的核心基础，必须得到足够重视。
- 一个成功的软件架构应具有：
 - 1) 良好的模块化
 - 2) 适应功能需求和技术的变化
 - 3) 对系统的动态运行有良好的规划
 - 4) 对数据的良好规划
 - 5) 明确、灵活的部署规划

17.1 引言 (2)

- 但是，任何软件架构在它的生命周期中随着软件系统的演化也会发生变化。
- 软件架构的变化存在两种情况：
 - 1) 软件架构向更好的方向变化，也就是说，软件架构在变化过程中，很多架构属性（特别是质量属性）会变好。
 - 2) 当软件系统经过一系列的演化之后，软件架构原来能够保持的一些软件架构属性（特别是质量属性）会变差、甚至不再保持了，这就是我们常说的，在软件演化过程中，软件架构会发生**腐蚀现象**。

17.2 软件架构腐蚀的含义

- 软件架构腐蚀 (software architecture erosion) 是指预期软件架构或概念软件架构与实际软件架构之间的偏离。
- 它意味着最终的实现并没有完全满足预定的计划或违背了系统的约束和规则。这种偏离更多的是源自日常的软件修改，而非人为的恶意。
- 架构腐蚀会导致软件演化过程中出现工程质量的恶化。

17.3 软件架构腐蚀的预防控制策略

- 三种预防控制方法
 - 腐蚀最小化
 - 腐蚀预防
 - 腐蚀修补

17.3.1 软件架构腐蚀最小化方法

- 腐蚀最小化方法的主要策略

(1) 面向过程的架构一致性策略
(Process-oriented architecture
conformance)

(2) 架构演化管理策略 (Architecture
evolution management)

(3) 架构设计实施策略 (Architecture
design enforcement)

17.3.1 软件架构腐蚀最小化方法

(1) 面向过程的架构一致性策略

- 一致性（conformance）对于减少软件架构腐蚀至关重要。
- 一致性常常通过以过程为核心的活动来实现。
- 诸多的文献表明**架构腐蚀与人为的因素有关**。
 - 设计文档匮乏
 - 设计规则的错误理解
 - 开发者的技术缺陷

17.3.1 软件架构腐蚀最小化方法

(1) 面向过程的架构一致性策略

- 一致性（conformance）对于减少软件架构腐蚀至关重要。
- 软件开发过程模型中往往都提倡要有效保障诸多的一致性问题的，例如：
 - 1) 架构要满足用户需求；
 - 2) 设计要符合架构指导；
 - 3) 变更需求必须经过审核；
 - 4) 修改必须符合预定规则。

17.3.1 软件架构腐蚀最小化方法

(1) 面向过程的架构一致性策略

- 软件工程过程采用以下一致性策略：
 - 统一而严格地编写**软件架构设计文档**。
 - 通过**软件架构分析**进行架构评估和复查。
 - **软件架构依从性监控**，使得实现忠于架构设计。
 - 通过**软件架构依赖性分析**揭示特定系统中违背软件架构规则的情况。

17.3.1 软件架构腐蚀最小化方法

(1) 面向过程的架构一致性策略

- 软件架构设计文档

- 糟糕的软件架构设计文档是导致软件架构腐蚀的主要原因之一。
- 为了更清晰地理解软件架构，文档必须足够详细地记录软件架构的分析和抽象。
- 成熟的软件过程要求文档必须集中记录系统的结构、与环境的交互等信息。
- 文档的描述方法，可分为形式化方法和非形式化方法。

17.3.1 软件架构腐蚀最小化方法

(1) 面向过程的架构一致性策略

- **软件架构分析**

- 软件架构分析主要采用的是架构评估和复查。
- ATAM允许软件架构师评估预期架构与用户需求、质量属性、设计决策的差别；
- SAAM方法分析体系结构的可修改性；
- 软件架构分析的其它各种方法.....

17.3.1 软件架构腐蚀最小化方法

(1) 面向过程的架构一致性策略

- **软件架构依从性监控**
 - 该策略包括常规的依从性监控活动，使得其后的实现忠于预先定义的架构。
 - 目前用于检测架构违规的技术：
 - 反射模型：反射模型（reflection model）技术用来比较架构师提供的高层模型与源代码实现的区别。商业化应用的反射模型工具包括 SAVE（Fraunhofer IESE）和 Structure-101（Headway Software）。
 - 特定领域语言工具：QL（Semmler Limited）和 DCL（Federal University of Minas Gerais）。

17.3.1 软件架构腐蚀最小化方法

(1) 面向过程的架构一致性策略

- **软件架构依赖性分析**

- 依赖性分析可以揭示特定系统中违背软件架构规则的情况。
 - 例如：一个严格的分层系统，每层都调用下一层的服务，任意一个代码构件如果没有遵从这个限制就视为违反了约定。
- Sangal等人提出了一个基于依赖矩阵的方法，该方法简单明了，且能够实现依赖性的自动分析。
- Hello2morrow公司则利用预期架构的XML配置文件来检测模块之间的依赖性。

17.3.1 软件架构腐蚀最小化方法

(1) 面向过程的架构一致性策略

策略	对控制腐蚀的意义
架构设计文档	记录软件设计过程，为软件的演化提供参考依据
架构分析	揭示预期架构的弱点，特别是在实现中容易被忽视的地方
架构依从性监控	验证架构是否满足依从性，即实现是否忠于预期的架构
依赖性分析	揭示架构之间的约束及依赖关系

17.3.1 软件架构腐蚀最小化方法

- 腐蚀最小化方法的主要策略

(1) 面向过程的架构一致性策略
(Process-oriented architecture
conformance)

(2) 架构演化管理策略 (Architecture
evolution management)

(3) 架构设计实施策略 (Architecture
design enforcement)

17.3.1 软件架构腐蚀最小化方法

(2) 架构演化管理策略 (Architecture evolution management)

- 架构演化管理试图将架构和系统看作一个整体，研究它们的共同演化，以此来达到控制架构腐蚀的目的。
- 软件配置管理 (Software configuration management, SCM) 工具可以控制、跟踪和记录架构及其实现的变化。

17.3.1 软件架构腐蚀最小化方法 (4)

(3) 架构设计实施策略 (Architecture design enforcement)

- 软件架构设计实施采用形式化或半形式化的方法对目标系统进行建模，以此来引导设计和编程活动。例如：模型驱动架构 (MDA)
- 其子策略包括：
 - 代码生成 (Code generation)
 - 架构风格/模式 (Architecture styles/patterns)
 - 架构框架 (Architecture frameworks) 等。

17.3 软件架构腐蚀的预防控制策略

- 三种预防控制方法
 - 腐蚀最小化；
 - 腐蚀预防；
 - 腐蚀修补；

17.3.2 腐蚀预防方法

• 联动架构策略

- 联动架构试图建立架构及其实现的连续关联。
- 这种关联一般是双向的，并且使得架构及其实现可以独立地演变，同时保证遵守预先定义的一致性规则。这样一来，与架构背离的实现将会被剔除，这也就达到了预防架构腐蚀的目的。
- 实施代价大，未广泛应用。

17.3.2 腐蚀预防方法

- **自适应策略**

- 软件架构腐蚀的原因，多是定期进行系统维护的人员没有严格遵守架构指南。
- 自适应系统一般是基于一个闭合反馈回路构成的传感器（或探针），检测系统的环境变化或其内部状态，并采用比较器（或计量表），评估传感器输出与预定义策略的不同。
- 自适应的有效性在很大程度上取决于设计师能否预见潜在的变化，而这在复杂性极高的软件系统中是非常困难的。

17.3 软件架构腐蚀的预防控制策略

- 三种预防控制方法
 - 腐蚀最小化；
 - 腐蚀预防；
 - 腐蚀修补；

17.3.3 腐蚀修补方法

- 架构腐蚀的控制方法旨在最小化和预防，并不能完全防止腐蚀。因此需要依赖修补方法的帮助来控制架构的腐蚀。
- 架构修补包括识别腐蚀、从源代码中恢复实现的架构、修复恢复的架构使之符合预期架构、协调实现与预期架构。

17.3.3 腐蚀修补方法

- 腐蚀修补方法(repair)包含三个主要策略：
 - 架构恢复策略 (Architecture recovery) :
见第十章
 - 架构发现策略 (Architecture discovery)
 - 架构协调策略 (Architecture reconciliation) 。

17.3.3 腐蚀修补方法 (3)

- 架构发现策略
 - 当预期架构的文档、规格不可用的时候，采用架构发现就显得非常必要了。
 - 架构的发现过程通过研究系统用例，从系统需求来构建预期的架构。

17.3.3 腐蚀修补方法

- 架构发现策略
 - Heimdhal等提出的方法：
 - (1) 通过研究系统用例，从系统需求来构建预期的架构。(架构发现)
 - (2) 从源码生成类图，对类图进行聚类抽取出另一个架构。(架构恢复)
 - (3) 将上述两个架构整合，生成一个可重用架构。
 - 架构发现本身并不足以修复架构腐蚀，它经常作为架构恢复和架构协调的辅助和补充手段。

17.3.3 腐蚀修补方法

- 架构协调策略

- 架构协调是修补实际架构以使其符合恢复或发现的预期架构的过程。
- 协调架构实现的一种常见方法是代码重构。遵循架构设计原则，源代码将被系统地重新组织，这一过程中不能改变系统的可见行为。
- 与逆向工程类似，代码重构也有一系列的工具来支持，得到广泛应用的工具包括Structure-101。
- 因为广泛的适用性和自动化，架构协调的方法已经在工业界得到应用。

17.3.3 腐蚀修补方法

策略	对控制腐蚀的意义
架构恢复	从源代码或其他架构文档中抽取架构，为腐蚀的检测、评价和修复建立基础
架构发现	当文档不可用时，从现有架构中发现新的架构以备重用
架构协调	修补实际架构以使其符合恢复或发现的预期架构

17.4 软件架构实践中面临的主要威胁及其对策

软件架构的设计异常重要，但是仅有成功的架构设计并不一定能够得到软件项目的成功，因为即使是好的架构在实际执行过程中也会遇到各种各样的困难，而结合了错综复杂的实际情况，使得进行架构设计也开始变得不那么容易。

17.5.1 主要威胁

- 产品线一致性威胁
 - 产品线和单品之间的架构不一致，产品线要先于单品开发，而单品因其个性化设计导致产品线架构的维护异常困难。
- 大规模软件架构腐蚀威胁
 - 大规模-» 复杂性-» 架构复杂-» 不断修改-» 架构混乱、不可理解，架构与代码的清晰对应关系
- 大开发团队中架构一致性威胁
 - 大型开发团队，异地开发，理解不同，沟通效果没有保障
- 旧产品软件架构缺失威胁
 - 遗产软件文档缺失或不准确

17.5.2 对策

根据上述威胁，软件架构在实践中遇到威胁的主要原因在于：

- (1) 架构方案对系统灵活性与可维护性的考虑不足；
- (2) 缺少来自于架构的足够的指导与限制；
- (3) 历史遗留问题。

17.5.2 对策

为应对前述威胁，在软件架构实践中，以‘实用主义’为指导，充分认识威胁所带来的风险，有针对性的做出决策是一种切实有效的方法。

另一方面，为了确保所设计的软件架构能够得到执行，开发团队思想一致性应得到更多的关注，可以采取一系列促进沟通或验证沟通效果的活动，从制度与技术两方面共同保障开发团队对架构的充分理解。

17.6 小结

- 软件架构腐蚀的概念
- 软件架构腐蚀的原因分析
- 软件架构腐蚀的预防控制方法